

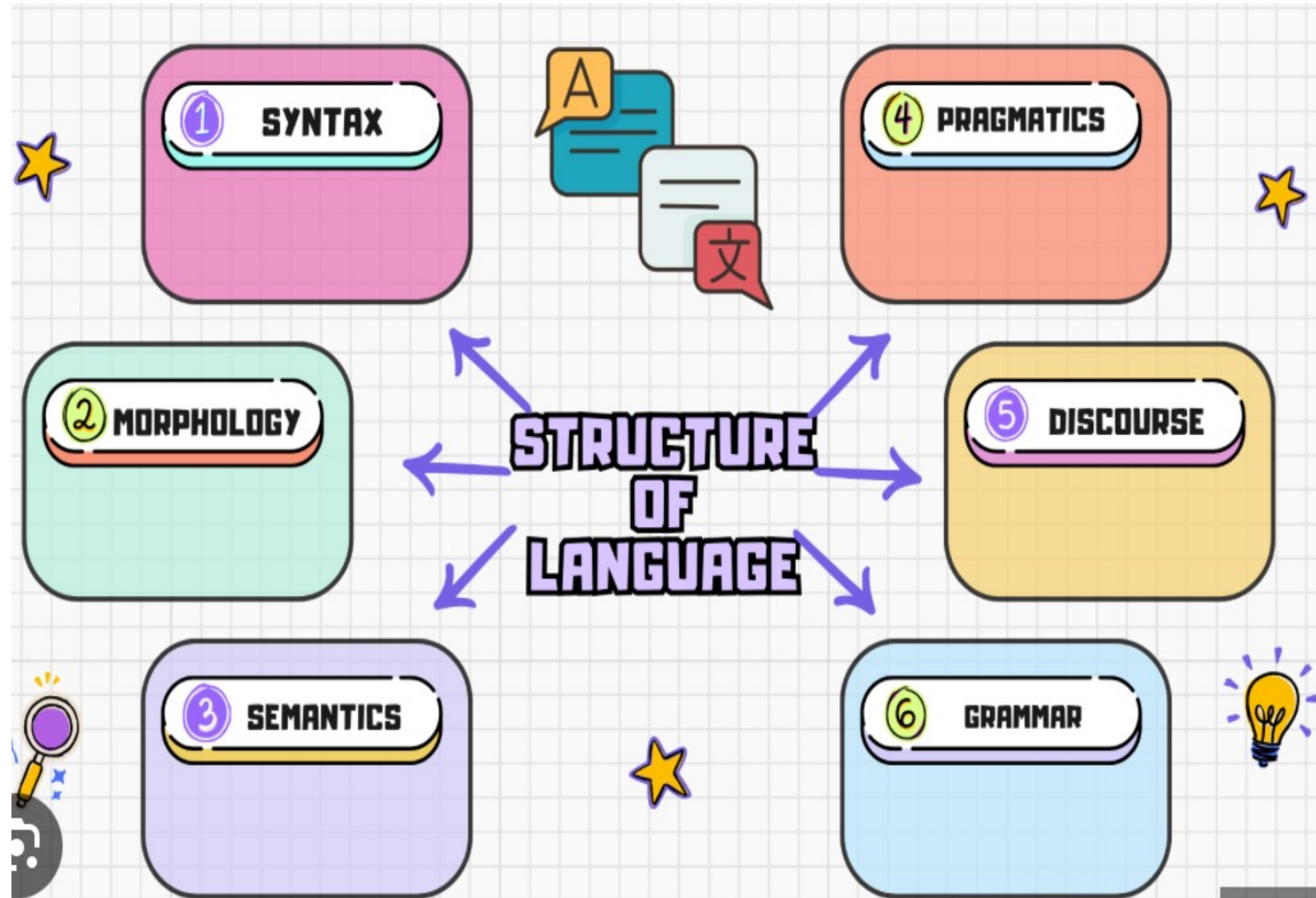
CO523 PL

-LANGUAGE STRUCTURE

-LANGUAGE PARADIGMS

Lec02-161225

★ PART 2: LANGUAGE STRUCTURE



The structure of a programming language is defined by its

- **syntax** (form) and **semantics** (meaning), which establish rules for organizing code into functional components and controlling program flow

Syntax vs. Semantics

Syntax: Structure/form (grammar rules)

Semantics: Meaning (behavior of constructs)

Examples:

- $x = y + 1 \rightarrow$ syntax rules
- $=$ means assignment $\rightarrow$ semantics

Lexical Elements

Tokens:

identifiers, keywords,
operators, punctuators

Lexical analyze converts stream of characters → tokens

Parsing

- Converts tokens → parse tree
- Grammar notation: BNF, EBNF

Example grammar for expressions

Parser:

A parser is a program that generates a parse tree for the given string, if the string is generated from the underlying grammar.

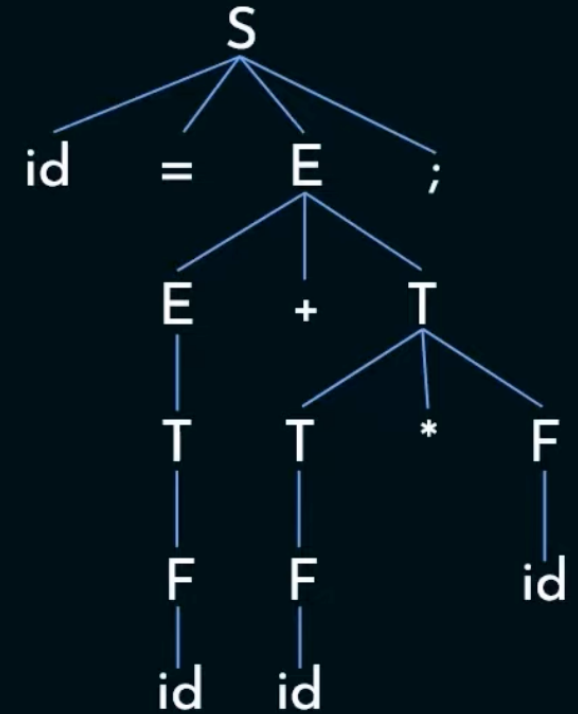
`id = id + id * id ;`



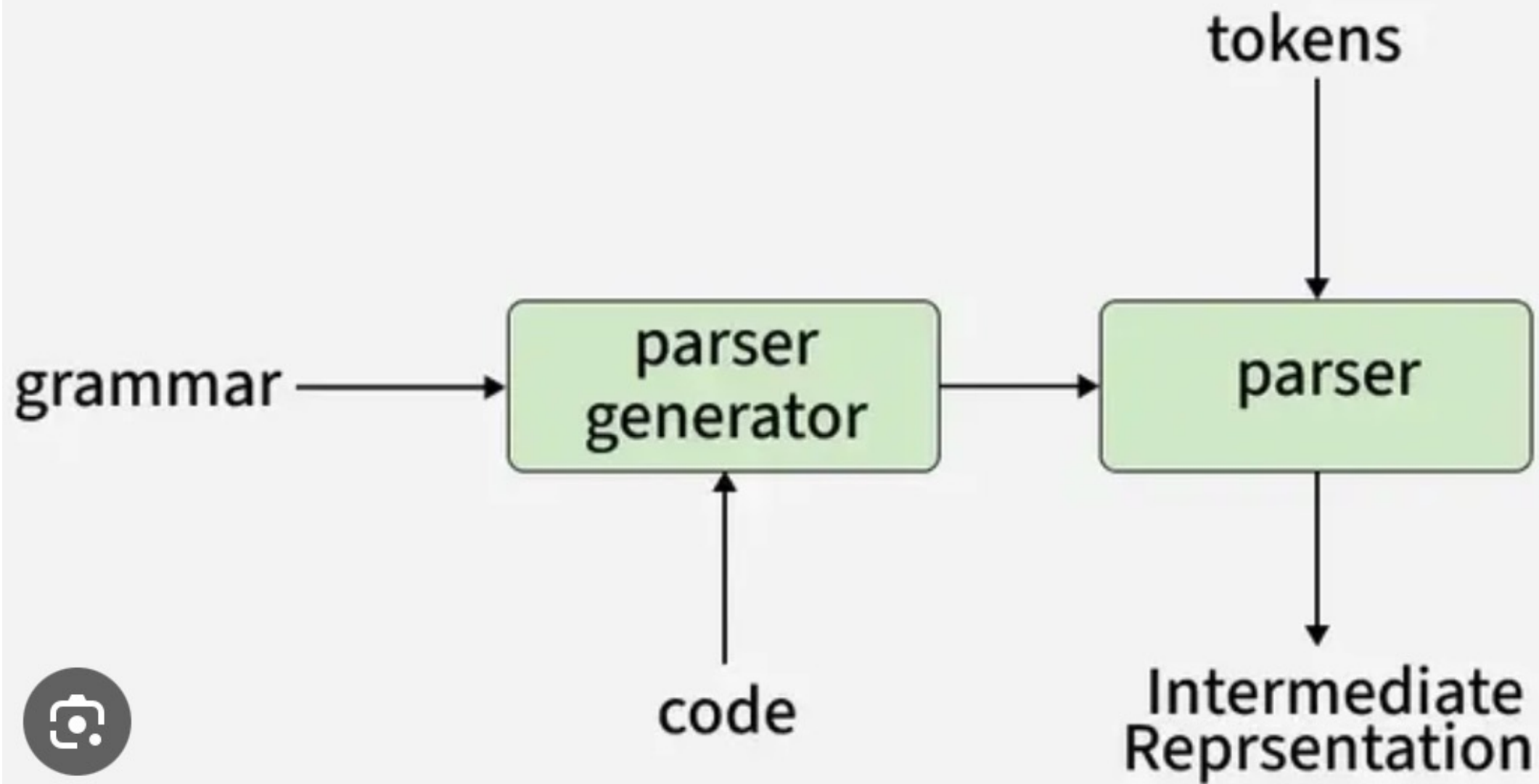
Parser



$S \rightarrow id = E ;$
 $E \rightarrow E + T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow id$



- In the syntax analysis phase, a compiler verifies whether or not the tokens generated by the lexical analyzer are grouped according to the syntactic rules of the language.
- This is done by a parser.
- It detects and reports any syntax errors and produces a parse tree from which intermediate code can be generated.



DEFINITION OF PARSING

- A parser is a compiler or interpreter component that breaks data into smaller elements for easy translation into another language.
- A parser takes input in the form of a sequence of tokens or program instructions and usually builds a data structure in the form of a parse tree or an abstract syntax tree.

Hello John *(input)*



Hello {name} *(model)*



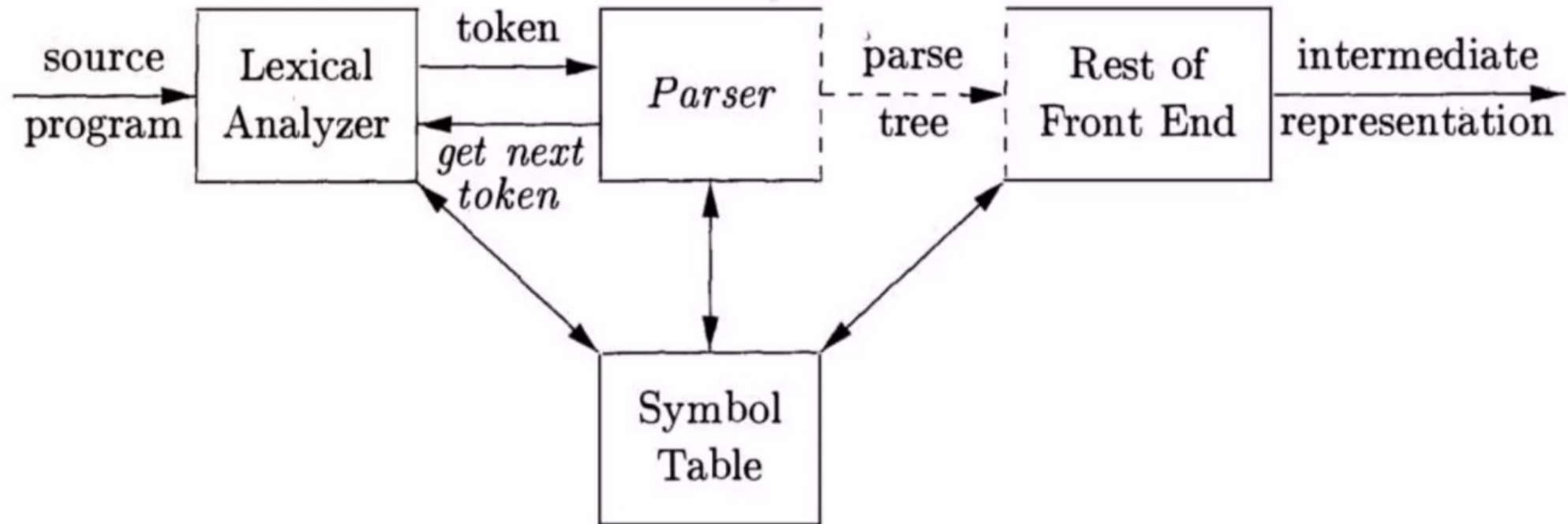
{name} = John *(data)*

What is file parsing

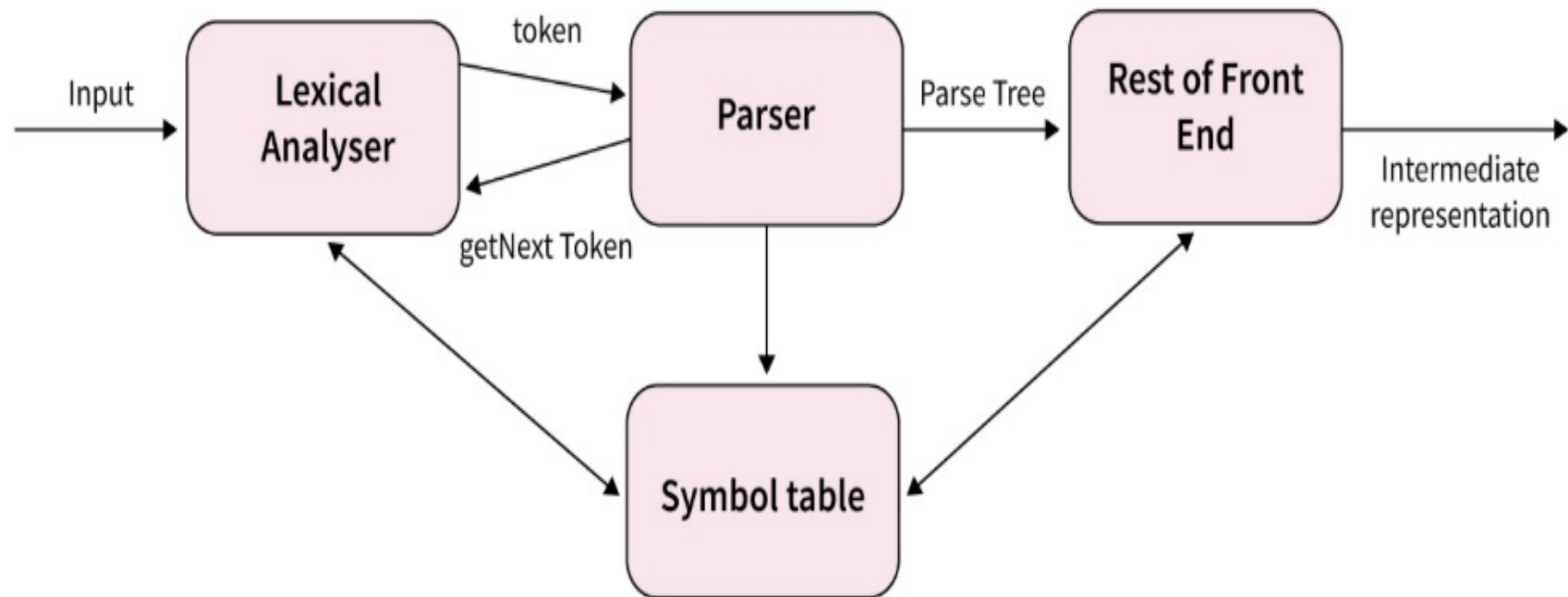


Parsing in Compiler Design

13



What is Parsing in NLP?



Context -free grammars

- **Terminals** - Basic symbols from which strings are formed.
- **Non-terminals** - Syntactic variables that denote sets of strings.
- **Start Symbol** - Denotes the nonterminal that generates strings of the languages
- **Productions** - $A = X \dots X$
 - Head/left side (A) is a nonterminal
 - Body/right side ($X \dots X$) zero or more terminals and non-terminals

- Non-terminals, terminals can be derived from productions.
- First production defines start symbol.

```

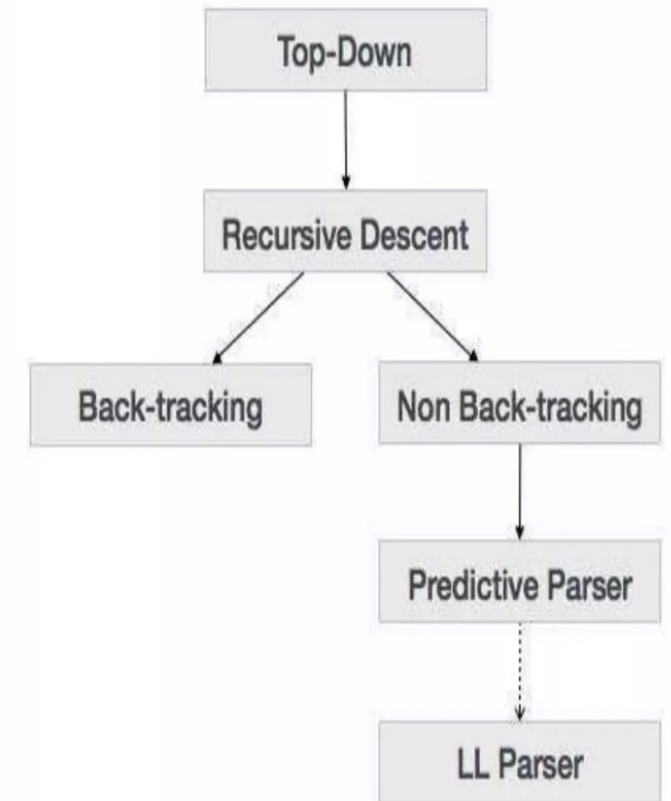
grammar
start S
non-terminals E T F
terminals "+" "*" "(" ")" ID
productions
  S = E
  E = E "+" T
  E = T
  T = T "*" F
  T = F
  F = "(" E ")"
  F = ID
  
```

Types of parser

- The process of deriving the string from the given grammar is known as derivation (parsing).
- Depending upon how derivation is done we have two kinds of parsers :-
- Top Down Parser
- Bottom Up Parser

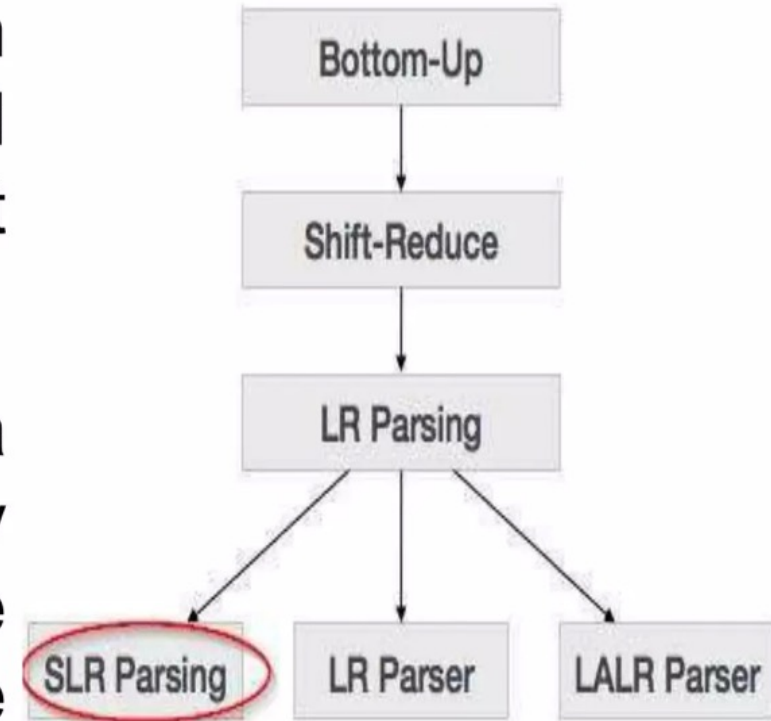
Top-down parser

- Top down parsing attempts to build the parse tree from root to leaf.
- Top down parser will start from start symbol and proceeds to string.
- It follows leftmost derivation.



Bottom-up parser

- Bottom-up parsing starts from the leaf nodes of a tree and works in upward direction till it reaches the root node.
- Here, we start from a sentence and then apply production rules in reverse manner in order to reach the start symbol.

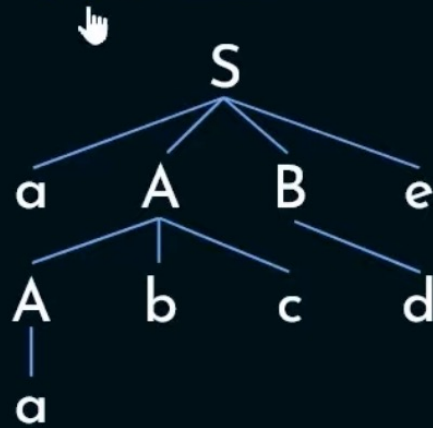


Generation of Parse Tree:

$S \rightarrow aABe, A \rightarrow Abc \mid a, B \rightarrow d$

aabcde

Top down approach:

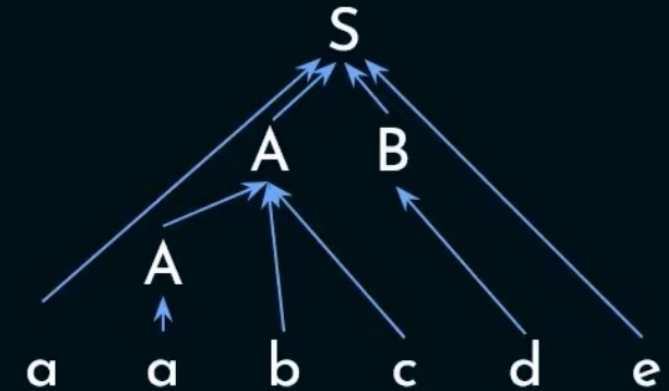


$S \Rightarrow aABe$
 $\Rightarrow aAbcBe$
 $\Rightarrow aabcBe$
 $\Rightarrow aabcde$

Decision:
Which production
to use.

(Left most Derivation)

Bottom up approach:

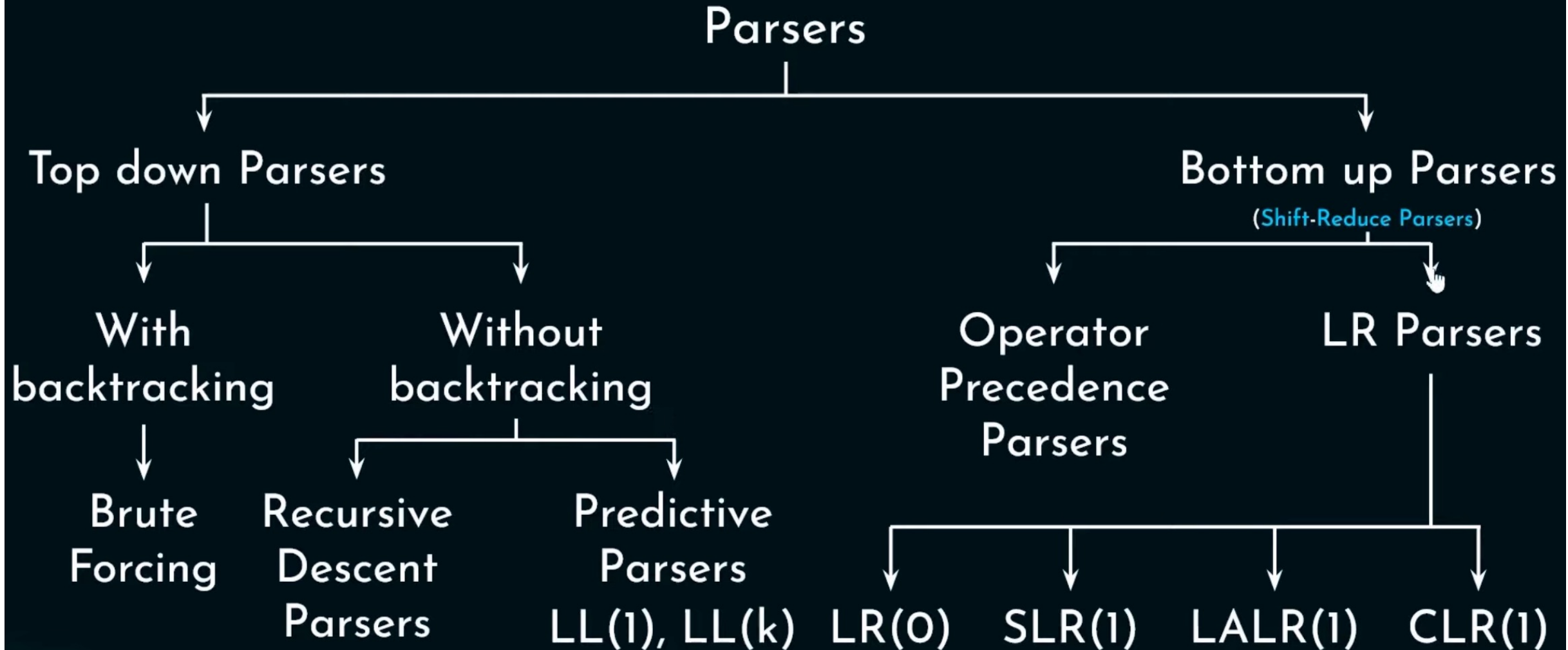


$S \Rightarrow aABe$
 $\Rightarrow aAde$
 $\Rightarrow aAbcde$
 $\Rightarrow aabcde$

Decision:
When to reduce.

(Right most Derivation) - In reverse.

Classification of Parsers:



Names, Variables, and Bindings



Name: symbolic identifier



Binding: association between name → value/type/location



Types of bindings:

Static

Dynamic

Explicit

Implicit

– Type Systems

Static typing: Java, C, Go

Dynamic typing: Python, JavaScript

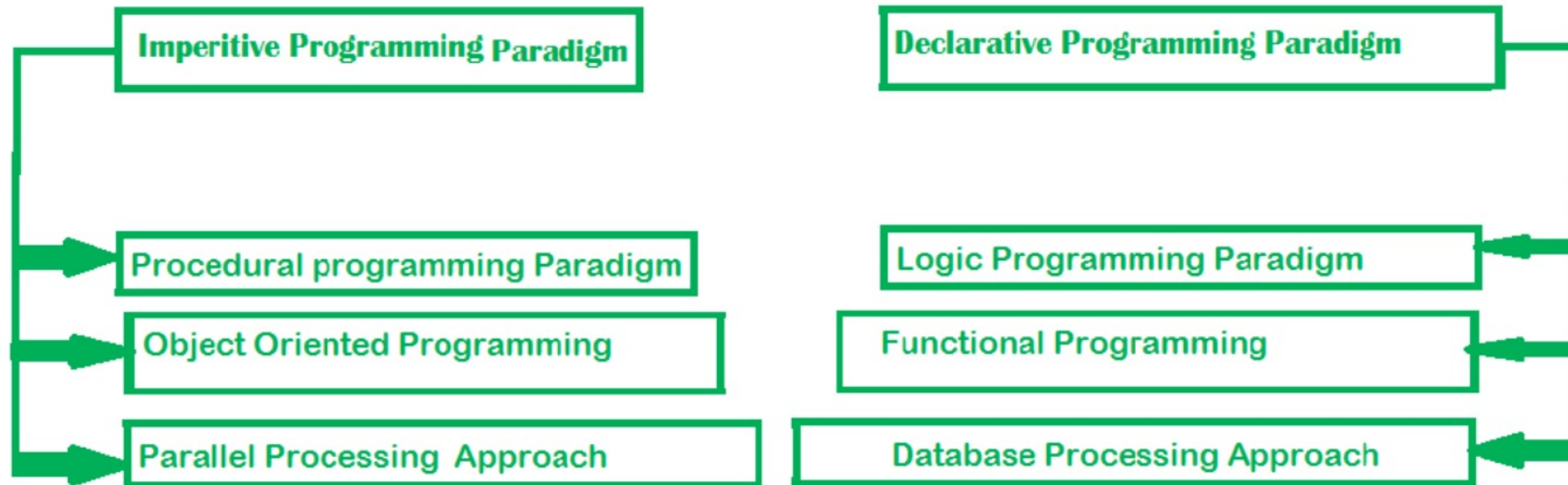
Strong vs. weak typing

Type inference (Haskell, Rust, Python 3.10+)

★ PART 3: LANGUAGE PARADIGMS

- A **paradigm** can also be termed as a method to solve a problem or accomplish a task. A **programming paradigm** is an approach to solving a problem using a specific programming language.

Programming Paradigms



1. Imperative Paradigm

- State changes
- Variables + assignments

Examples: C, Python, Java

1. Imperative programming paradigm: It features close to machine architecture. It is based on Von Neumann architecture.

It works by changing the program state through assignment statements. It performs step by step task by changing state. The main focus is on how to achieve the goal.

Advantages:

1. Very simple to implement
2. It contains loops, variables etc.

Disadvantage:

1. Complex problem cannot be solved
2. Less efficient and less productive
3. Parallel programming is not possible

Examples of **Imperative** programming paradigm:

C : developed by Dennis Ritchie and Ken Thompson

Fortran : developed by John Backus for IBM

Basic : developed by John G Kemeny and Thomas E Kurtz

```
#include <iostream>

int main() {
    // Array to store marks
    int marks[5] = { 12, 32, 45, 13, 19 };

    // Variable to store the sum of marks
    int sum = 0;

    // Variable to store the average
    float average = 0.0;

    // Calculate the sum of marks
    for (int i = 0; i < 5; i++) {
        sum = sum + marks[i];
    }

    // Calculate the average
    average = sum / 5.0;

    // Output the average
    std::cout << "Average of five numbers: " << average << std::endl;

    return 0;
}
```

Average of five numbers: 24.2

- Imperative programming is divided into three broad categories: **Procedural, OOP and parallel processing.**
- Procedural programming paradigm - emphasizes on procedure in terms of underlying machine model.

here is no difference in between procedural and imperative approach. It has the ability to reuse the code and it was boon at that time when it was in use because of its reusability.

Examples of **Procedural** programming paradigm:

C : developed by Dennis Ritchie and Ken Thompson

C++ : developed by Bjarne Stroustrup

Java : developed by James Gosling at Sun

Microsystems

ColdFusion : developed by J J Allaire

Pascal : developed by Niklaus Wirth

example

```
#include <iostream>
using namespace std;
int main()
{
    int i, fact = 1, num;
    cout << "Enter any Number: ";
    cin >> number;
    for (i = 1; i <= num; i++) {
        fact = fact * i;
    }
    cout << "Factorial of " << num << " is: " << fact << endl;
    return 0;
}
```

Object oriented programming -

- The program is written as a collection of classes and object which are meant for communication.
- The smallest and basic entity is object and all kind of computation is performed on the objects only. More emphasis is on data rather procedure. It can handle almost all kind of real life problems which are today in scenario.

Advantages:

- Data security
- Inheritance
- Code reusability
- Flexible and abstraction is also present

– Object-Oriented Paradigm

Classes and objects

Encapsulation

Inheritance

Polymorphism

Examples: Java, C++, Python

- Examples of **Object Oriented** programming paradigm:
 - Simula** : first OOP language
 - Java** : developed by James Gosling at Sun Microsystems
 - C++** : developed by Bjarne Stroustrup
 - Objective-C** : designed by Brad Cox
 - Visual Basic .NET** : developed by Microsoft
 - Python** : developed by Guido van Rossum
 - Ruby** : developed by Yukihiro Matsumoto
 - Smalltalk** : developed by Alan Kay, Dan Ingalls, Adele Goldberg

Parallel processing approach -

Parallel processing is the processing of program instructions by dividing them among multiple processors. A parallel processing system possesses many numbers of processor with the objective of running a program in less time by dividing them. This approach seems to be like divide and conquer. Examples are NESL (one of the oldest one) and C/C++ also supports because of some library function.

2. Declarative programming paradigm:

It is divided as Logic, Functional, Database.

In computer science the *declarative programming* is a style of building programs that expresses logic of computation without talking about its control flow.

It often considers programs as theories of some logic.

It may simplify writing parallel programs. The focus is on what needs to be done rather how it should be done basically emphasize on what code is actually doing.

It just declares the result we want rather how it has be produced. This is the only difference between imperative (how to do) and declarative (what to do) programming paradigms.

Functional programming paradigms -

- The functional programming paradigms has its roots in mathematics and it is language independent.
- The key principle of this paradigms is the execution of series of mathematical functions.
- The central model for the abstraction is the function which are meant for some specific computation and not the data structure. Data are loosely coupled to functions.
- The function hide their implementation. Function can be replaced with their values without changing the meaning of the program. Some of the languages like perl, javascript mostly uses this paradigm.

- Examples of **Functional** programming paradigm:
 - JavaScript** : developed by Brendan Eich
 - Haskell** : developed by Lennart Augustsson, Dave Barton
 - Scala** : developed by Martin Odersky
 - Erlang** : developed by Joe Armstrong, Robert Virding
 - Lisp** : developed by John McCarthy
 - ML** : developed by Robin Milner
 - Clojure** : developed by Rich Hickey

– **Functional Paradigm**

Functions as first-class citizens

No side effects

Recursion instead of loops

Examples: Haskell, Lisp, Scala

Logic programming paradigms -

- It can be termed as abstract model of computation. It would solve logical problems like puzzles, series etc. In logic programming we have a knowledge base which we know before and along with the question and knowledge base which is given to machine, it produces result.
- In normal programming languages, such concept of knowledge base is not available but while using the concept of artificial intelligence, machine learning we have some models like Perception model which is using the same mechanism.
- In logical programming the main emphasize is on knowledge base and the problem. The execution of the program is very much like proof of mathematical statement, e.g., Prolog

– Logic Programming

- Rule-based
- Uses unification and backtracking
- Example: Prolog

Database/Data driven programming approach -

- This programming methodology is based on data and its movement. Program statements are defined by data rather than hard-coding a series of steps.
- A database program provides file creation, data entry, update, query and reporting functions.

For example SQL. It is applied to streams of structured data, for filtering, transforming, aggregating (such as computing statistics), or calling other programs. So it has its own wide application.

Concurrent

A concurrent programming paradigm is

- a way of structuring programs where multiple tasks make progress seemingly simultaneously, overlapping execution to use multi-core CPUs, improve responsiveness, and handle I/O efficiently, contrasting with sequential code by allowing non-deterministic, interleaved operations, managed via models like **Threads & Locks, Message Passing, or Actor Models**, dealing with complexities like shared state and race conditions.

Programming Paradigms

